Project: Authentication & API Gateway

Due: 03/21/2026 11:59 PM
Extended Deadline: 03/22/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Ingress + Gateway
- Task Overview
- Task

KrakenD
- Task Overview
- Task

Django Authentication
- Task Overview
- Task

Securing LLM API
- Task Overview
- Task

Wrap Up
- Conclusion

Securing LLM API (Task)

Project Architecture

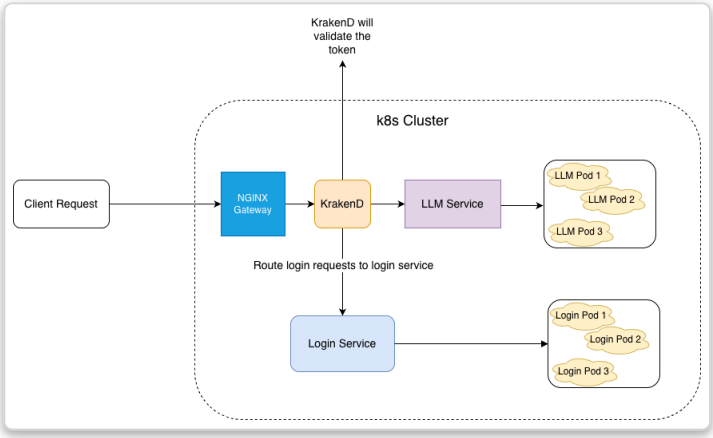


Figure: Task 4 Architecture Diagram

Take some time to review the diagram and familiarize yourself with each component. Understanding how these pieces work together will make it much easier to follow the remaining steps of the project.

In the previous task, we focused on building an authentication service using Django. This service exposes a single endpoint, `/login`. When valid user credentials are sent to this endpoint, it returns a login token, also known as an access token. The client can then use this access token to call protected resources.

However, we didn't discuss how those protected resources are actually secured. That's what we'll cover in this task.

A straightforward approach you might think of is to copy the LLM-related code directly into the Django app and add a simple authentication check, something like this in pseudocode:

```
if user.is_authenticated:
    return LLMResponse()
else:
    raise HTTPException(status_code=401)
```

What this logic does is fairly simple: it checks whether the user is authenticated by verifying that an access token was provided in the request header and that the token is valid. We'll discuss how token validation works in more detail later. While this approach is easy to understand and visualize, it introduces a few problems when you try to scale it.

First, the login service is stateful because it relies on a database to store and retrieve user information. This makes horizontal scaling more complex compared to a stateless service.

Second, this approach effectively turns the system into a monolith. Since authentication and core application logic live in the same service, we lose the benefits of separation. We no longer have a cleanly isolated LLM service, and scaling or extending the system with additional endpoints becomes harder.

So let's look at a better approach.

What we really want is this:
The login service issues a token → the client sends that token with every request → the gateway (KrakenD) validates the token → only then is the request forwarded to the LLM service.

In this setup, the LLM app remains completely unchanged. It does not need to know anything about authentication. The gateway, which is already designed to handle scale and traffic management, takes care of validation. This makes the system cleaner, more scalable, and easier to extend with additional microservices in the future.

Now the question is: how does KrakenD know whether a token is valid?

One option is to ask the login service every time. But that would quickly turn the login service into a bottleneck and introduce unnecessary network overhead.

This is where cryptography — and JWTs — come into play.

JWT (JSON Web Token) is a format that allows anyone to read the contents of the token, but prevents anyone from modifying it. Only the entity that possesses the correct cryptographic key can generate or tamper with a valid token.

So far, we've been using a symmetric signing approach, where the same secret key is used both to sign and verify the token. That works, but it means KrakenD would need access to the same secret which is not ideal.

A better approach is to use asymmetric cryptography.

With asymmetric cryptography, we use a key pair:

A **private key**, held only by the login service
A **public key**, which can be shared safely

The login service signs the token using the private key. Any service that has the corresponding public key such as KrakenD can verify the token's authenticity without needing access to the private key. This allows the gateway to validate tokens independently, keeps the login service from becoming a bottleneck, and maintains strong security guarantees.

This design keeps the LLM service stateless, improves scalability, and cleanly separates authentication from business logic.

Finally, here is the outline for this task:

1. Read and understand the KrakenD requirements for enabling authentication.
2. **Subtask 1** – Update the Django application to use an asymmetric algorithm and implement the necessary changes to make it compatible with the format expected by KrakenD.
3. **Subtask 2** – Deploy the updated service.
4. **Subtask 3** – Update KrakenD to enable authentication and perform JWT validation.

Information

This task is fairly extensive and may involve a good amount of debugging. We recommend taking a break after completing Subtask 3 before continuing further. Starting fresh would surely make this task much easier and more efficient.

Understanding KrakenD Requirements

Every API gateway has its own way of handling authentication and validation. In general, you will always find two core components involved:

- A **token validator**, which uses a public key to verify the token
- An **identifier**, which maps a token to the correct public key in cases where multiple token issuers exist

The entity responsible for generating the token is known as the **Identity Provider**. In our case, this role is played by the **login service**.

KrakenD has two key requirements for JWT-based authentication:

1. **Each JWT must include a key identifier (`kid`)** in its header.
   This allows KrakenD to determine which public key should be used to validate the token.
2. **Every protected endpoint must specify a JWK path (or JWK URL).**
   This path exposes the public key used to verify the token signature. The JWK endpoint must return a JSON object that:
   - Contains the exact `kid` present in the JWT header
   - Provides the public key in JWK format

   KrakenD uses the `kid` value to select the correct key when validating a request. Multiple keys can exist in the JWK response, allowing support for key rotation or multiple token issuers.

In our case, we will provide this JWK JSON as a Kubernetes secret.

Below is an example of the expected JWK format:

```
{
  "keys": [
    {
      "kty": "RSA",
      "n": "<public_key>",
      "e": "AQAB",
      "kid": "<kid>",
      "alg": "RS256"
    }
  ]
}
```

This structure allows KrakenD to securely verify JWTs without needing access to the private signing key, keeping the authentication flow both scalable and secure.

Subtask 1: Modifying the Django Application

You can run the Django application from the previous task and paste the generated token into https://jwt.io/. You should see the token split into three parts: **header**, **payload**, and **signature**. Right now, the algorithm we're using (shown in the `"alg"` field of the JWT header) is **HS256**.

To decouple signing and verification—and make this more production-grade—we'll switch to **RS256**, an asymmetric signing standard. Unlike HS256, which uses a single shared secret, RS256 uses two keys:

A **private key**, kept securely within the application and used to **sign** tokens
A **public key**, shared with tools like KrakenD and used only to **verify** tokens

This lets KrakenD validate tokens without needing access to any sensitive signing secret.

Generating RSA Keys

To use RS256, we first need a private/public key pair. Run the following commands in the project root folder:

```
openssl genpkey -algorithm RSA -out privat
openssl rsa -pubout -in private_key.pem -o
```

The first command generates the private key, and the second extracts the public key from it.

Make sure you add both key files to your `.gitignore` so you don't accidentally commit them.

Loading Keys Using Environment Variables

Now that you have the keys, we need a way for the Django application to access them. The recommended approach is to use environment variables:

```
export PUBLIC_KEY="$(cat public_key.pem)"
export PRIVATE_KEY="$(cat private_key.pem)
```

Next, in `settings.py`, add the following at the end:

```
SIMPLE_JWT = {
    "ALGORITHM": TODO1,
    "SIGNING_KEY": TODO2,
    "VERIFYING_KEY": TODO3,
}
```

You'll need to set:

The algorithm
The signing key (private key)
The verifying key (public key)

Refer to the documentation here for the correct values: https://django-rest-framework-simplejwt.readthedocs.io/en/latest/settings.html

As a hint: you can read environment variables in Python by importing `os` and using `os.getenv("MY_VARIABLE")`.

Adding a `kid` Header to the Tokens

Next, we'll modify the Django views so the JWT tokens include a `kid` field in the header. For this project, we'll set the `kid` value to `mycoolkid`.

Start by adding the following class-based view to `task3/login/views.py`:

```
class MyTokenObtainPairView(TokenObtainPai
    def post(self, request, *args, **kwarg
        serializer = self.get_serializer(d

        try:
            serializer.is_valid(raise_exce
        except TokenError as e:
            raise InvalidToken(e.args[0])

        # re-write the access token to mak
        def add_kid_header(mat) -> dict:
          """Add kid header to token

            Args:
                mat (str): Encoded headers

            Returns:
                jwt: Encoded new jwt.
            """
            # TODO 1 — Decode the token in
```

```
            # You need to decode the incom
            # The encoded token
            # The verifying key — hint: it
            # The algorithm — also inside :

            payload =

            # TODO 2 — Re-encode the paylo
            # Now encode the payload using

            # The payload you just decoded
            # The signing key — check SIMP
            # The algorithm — this time ar
            # A headers argument — a dicti

            new_token =

            return new_token

        serializer.validated_data['access'
        serializer.validated_data['refresh

        return Response(serializer.validat
```

Then add the required imports:

```
from rest_framework_simplejwt.views import
from rest_framework.response import Respon
from rest_framework import status
from rest_framework_simplejwt.exceptions i
import jwt
from django.conf import settings
from task3.settings import SIMPLE_JWT
```

Finally, install the `pyjwt` library using Poetry. You'll use it to update the token headers.

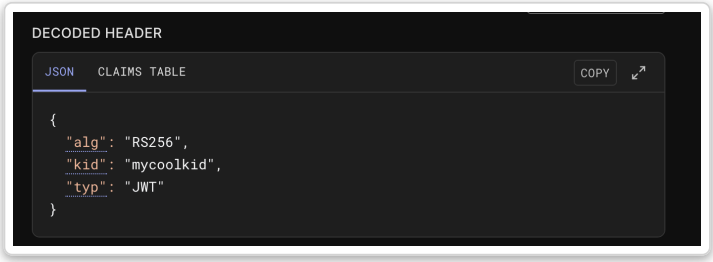
Your Task

Complete the `add_kid_header` function so it returns a JWT token that includes a `kid` header set to `mycoolkid`. You must use the PyJWT documentation to implement this correctly.

Once done, update `urls.py` so the `'api/token/'` route uses `MyTokenObtainPairView` instead of the default token view.

Validation

Your `/api/token/` endpoint must return a valid **access** and **refresh** token. The access token must include a `kid` field in the JWT header. You can verify this by pasting the token into jwt.io and inspecting the header section.



Subtask 2: Deploying the Login Service

Now that you've verified your JWT token is valid, we can deploy the login service to Kubernetes.

We'll begin by installing a Poetry plugin that makes it easy to generate a `requirements.txt` file from your locked dependencies. This `requirements.txt` can then be used to install the required packages inside your Docker image.

`poetry-plugin-export` is a handy plugin that allows you to export locked packages to different formats.

Install it using:

```
poetry self add poetry-plugin-export
```

Then, from inside your `django-authentication` folder, run:

```
poetry export -f requirements.txt -o requi
```

This generates the `requirements.txt` file.

Project Structure

Inside the `django-authentication` folder, create two directories:

```
docker/
k8s/
```

Your task is to create a **Dockerfile** and the **Kubernetes manifests** for the login service.

Dockerfile Tips

Here are a few tips to guide your Dockerfile:

1. For this task, we recommend using the Django development server since it makes debugging much easier.
2. For your `CMD`, use something like:
   `python manage.py runserver 0.0.0.0:8080`
   Using `0.0.0.0` ensures the service is reachable outside Docker and outside the pod.
3. Make sure you copy `db.sqlite3` into the container. This isn't production best practice, but it's fine for this project since the focus is on service integration with KrakenD.
4. We are using `python:3.12-alpine` as the base image.

Once your Docker image builds and runs correctly, log into your ACR and push the image:

```
az acr login -n <acr_name>
docker push <acr_name>.azurecr.io/loginapp
```

Deploying to Kubernetes

Next, deploy the container to Kubernetes by writing a **Deployment** and a **Service**.

Before doing that, we'll provide the RSA keys to the pod using a Kubernetes Secret. Run the following command from the directory where your `.pem` files are present:

```
kubectl create secret generic loginkeys --
```

Now create `deployment.yaml` and `service.yaml` inside the `django-authentication/k8s` folder.

Service Configuration

Name the service: `loginservice`

Use port **80** for external connections

Use port **8080** for pod connections

Set the service type to **LoadBalancer** (this helps with troubleshooting if KrakenD integration fails)

Deployment Configuration

Name the deployment: `login`

Run it as a **single pod** (replicas = 1)

Scaling is not covered yet, since this is meant to be a simple deployment focused on a stateful service.

Environment Variables

Use a Kubernetes Secret to supply the public and private keys as environment variables.

You can refer to this link for examples of how to load a secret into environment variables inside a Deployment.

Information

In the `deployment.yaml` file, you need to define two environment variables: `PUBLIC_KEY` and `PRIVATE_KEY`.

Think carefully about where their values come from. You already created a Kubernetes secret called `loginkeys` with this command:

```
kubectl create secret generic loginkeys --f
```

You can verify the secret exists and contains the right data by running:

```
kubectl describe secret loginkeys
```

You should see something like this:

```
Name:         loginkeys
Namespace:    default
Type:         Opaque

Data
====
private_key.pem:  1704 bytes
public_key.pem:   451 bytes
```

So the question is: how do you reference those secret values in your `deployment.yaml`?

Think about it this way:

> What is the **secret name**?
> What are the **keys** inside that secret?
> How does Kubernetes let you pull a value from a secret into an environment variable?

Once you figure that out, your `deployment.yaml` env section should load `private_key.pem` into `PRIVATE_KEY` and `public_key.pem` into `PUBLIC_KEY`.

Interim Validation

Once deployed, confirm that the LoadBalancer endpoint for `loginservice` can generate access tokens using the credentials we've been using:

Username: `admin`
Password: `asdasd`

Use the same curl command from the previous task:

```
curl \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"username":"admin", "password":"asd
  http://<load_balancer_ip>:<port>/api/tok
```

Subtask 3: Updating `krakend.json`

If you take a step back and look at the overall flow, we now have a service that generates JWT tokens with a `kid` header, and we also have a public key that can be used to verify those tokens. That's half the work done. The remaining step is to tell KrakenD which public key it should use to perform JWT validation.

This subtask can be broken into two parts:

1. Configure KrakenD to protect the LLM endpoint, and add the login service to KrakenD.
2. Add JWKS-based validation using a Kubernetes Secret.

Update the KrakenD Config in KrakenD Designer

Open the KrakenD Designer and load your existing config file. Make the following changes:

1. Add `http://loginservice` to the list of **Hosts** on the **Service Settings** page.
2. Add a new route `/v1/logintoken` that:
   - Accepts a `POST` request
   - Points to `http://loginservice/api/token/`
   - Accepts **all headers** (to keep things simple)
3. Go back to the `/v1/llm` route and open the **Auth** tab at the top:
   - Enable **token validation**
   - Turn **Disable JWK security** on
   - Set the algorithm to **RS256**

Download the updated `krakend.json` and upload it to your student work VM.

Update the YAML and Add JWKS as a Kubernetes Secret

After uploading the updated `krakend.json`, you'll need to make a few additional changes in the YAML. Some useful debugging and deployment options are not available through KrakenD Designer, so we'll add them manually.

Under the `auth/validator` section in your KrakenD config, make sure you have the following fields:

```
"alg": "RS256",
"jwk_local_path": "/etc/krakensecrets/publ
"disable_jwk_security": true,
"operation_debug": true
```

Once you've made these changes, confirm that the `krakend.json` validates using the KrakenD CLI.

Converting the Public Key to JWK

Next, we need to convert the public key into JWK format. Here is a one-liner in Python to do that:

```
cat <path_to_public_key.pem> | poetry run
```

Make sure you provide the correct path for `public_key.pem`. You should see output like:

```
{
  "kty": "RSA",
  "n": "...",
  "e": "AQAB",
  "alg": "RS256",
  "kid": "mycoolkid"
}
```

Now save this output into a separate file (`jwk.json`), but wrapped in the JWKS structure KrakenD expects:

```
{
  "keys": [
    {
```

```
      "kty": "RSA",
      "n": "...",
      "e": "AQAB",
      "alg": "RS256",
      "kid": "mycoolkid"
    }
  ]
}
```

Storing the JWKS JSON as a Kubernetes Secret

You'll be making your JWKS public key available to the KrakenD pod as a file at `/etc/krakensecrets/publickey.json`.

This involves 3 steps:

Step 1 — Convert your `jwk.json` to base64

```
base64 -w 0 jwk.json
```

Copy the output, you'll need it in the next step.

Step 2 — Create the secret

Create a file called `secret.yaml` in the **k8s-components** directory with the following content:

```
apiVersion: v1
kind: Secret
metadata:
  name: jwks-secret
type: Opaque
data:
  publickey.json: <paste-base64-output-her
```

> Notice the key is named `publickey.json` — this will become the filename inside the pod.

Apply it:

```
kubectl apply -f secret.yaml
```

Verify it was created:

```
kubectl describe secret jwks-secret
```

You should see `publickey.json` listed under `Data`.

Step 3 — Mount the secret into the KrakenD pod

In your `api-gateway.yaml`, add a volume and mount it into the container:

```
volumes:
  - name: jwks-volume
    secret:
      secretName: jwks-secret

volumeMounts:
  - name: jwks-volume
    mountPath: /etc/krakensecrets
```

Information

The above configuration is similar to what was done for krakend-cfg.

The secret will now be available inside the container at `/etc/krakensecrets/publickey.json`.

> **Think of it this way:** the `volume` attaches the secret to the pod, and the `volumeMount` tells the container where to find it.

Since you updated `krakend.json`, you'll also need to delete and recreate the `krakend-cfg` ConfigMap. Once that's done, delete the KrakenD pods so the new pods come up with:

The updated KrakenD config
The mounted JWKS secret

Interim Validation

Before you try curling the LLM endpoint, make sure your KrakenD logs look like this:

```
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
```

```
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND DEBUG: [BACKEN
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND DEBUG: [ENDPOI
2026/03/04 02:46:04 KRAKEND INFO: [ENDPOIN
```

Also, make sure you have updated your `httproute.yaml` to accept traffic for the endpoint `/v1/logintoken`. Don't forget to apply your configuration.

Check that you can acquire a token using:

```
curl -H 'Content-Type: application/json' -
```

If you run into issues, you can temporarily change the KrakenD service type to `LoadBalancer` and debug from there.

Also check that the `loginservice` is healthy and inspect the KrakenD pod logs for useful error messages. If you still can't resolve the issue, feel free to post on the discussion platform. We expect most of the tricky issues are covered, but it's possible something was missed during testing.

Calling the LLM Endpoint (Only After Token Flow Works)

If (and only if) the token flow works, you can then call the LLM API.

First, fetch an access token:

```
curl -H 'Content-Type: application/json' -
```

Then, replace `InsertACCESSTOKEN` with the access token and run:

```
curl -D - -H "Authorization: BEARER Insert
```

If everything is configured correctly, you should get a response from the LLM endpoint.

Final Cleanup

Finally, switch the services back to their intended types:

Change the KrakenD service from **LoadBalancer** back to **ClusterIP**
Change the login service from **ClusterIP** to **LoadBalancer**

At the end, the full login flow and the LLM endpoint must work on your domain over HTTPS.

Submission

k8s Components Tree Structure

```
k8s-components/
├── api-gateway.yaml
├── clusterissuer.yaml
├── gateway.yaml
├── httproute.yaml
├── krakend.json
└── secret.yaml

1 directory, 6 files
```

django-authentication Tree Structure

```
django-authentication/
├── Makefile
├── db.sqlite3
├── docker
│   └── Dockerfile
├── k8s
│   ├── deployment.yaml
│   └── service.yaml
├── manage.py
├── poetry.lock
├── poetry.toml
├── pyproject.toml
├── requirements.txt
└── task3
    ├── __init__.py
    ├── __pycache__
    │   ├── __init__.cpython-312.pyc
    │   ├── settings.cpython-312.pyc
    │   ├── urls.cpython-312.pyc
    │   └── wsgi.cpython-312.pyc
    ├── asgi.py
    ├── login
    │   ├── __init__.py
```

```
│   ├── __pycache__
│   │   ├── __init__.cpython-312.pyc
│   │   ├── serializers.cpython-312.p
│   │   └── views.cpython-312.pyc
│   ├── admin.py
│   ├── apps.py
│   ├── migrations
│   │   └── __init__.py
│   ├── models.py
│   ├── serializers.py
│   ├── tests.py
│   └── views.py
├── settings.py
├── urls.py
└── wsgi.py

8 directories, 30 files
```

Cluster State

```
azureuser@qa-vm:~/cloud-native-handout-mik
NAME                                  READY
krakend-deploy-896645f9c-v4sbw        1/1
llmapp-deployment-794767976f-gzzb4    1/1
login-5dc68c5f9-gl4g7                 1/1

azureuser@qa-vm:~/cloud-native-handout-mik
NAME              TYPE           CLUSTER-IP
apigateway        ClusterIP      10.0.13.10
kubernetes        ClusterIP      10.0.0.1
llmapp-service    ClusterIP      10.0.101.2
loginservice      LoadBalancer   10.0.237.1

azureuser@qa-vm:~/cloud-native-handout-mik
NAME        HOSTNAMES                      
llmroute    ["s26cloudnative.myvnc.com"]

azureuser@qa-vm:~/cloud-native-handout-mik
NAME                DATA   AGE
krakend-cfg         1      2d16h
kube-root-ca.crt    1      5d20h

azureuser@qa-vm:~/cloud-native-handout-mik
NAME                   TYPE                
jwks-secret            Opaque
listener-tls-secret    kubernetes.io/tls
```

```
loginkeys              Opaque
```

KrakenD logs

If you setup your infrastructure correctly, then logs from your Krakend pod should look like below. Pay attention to the last record where a request was made via the llm service.

```
kubectl logs krakend-deploy-896645f9c-v4sb

2026/03/06 18:46:37 KRAKEND INFO: Starting
2026/03/06 18:46:37 KRAKEND INFO: Working
2026/03/06 18:46:37 KRAKEND DEBUG: [SERVIC
2026/03/06 18:46:37 KRAKEND INFO: Starting
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [BACKEN
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [BACKEN
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND DEBUG: [ENDPOI
2026/03/06 18:46:37 KRAKEND INFO: [ENDPOIN
2026/03/06 18:46:37 KRAKEND INFO: [SERVICE
2026/03/06 18:46:42 KRAKEND DEBUG: [SERVIC
[GIN] 2026/03/06 - 19:31:20 | 200 | 11.134
```

Run the submitter to finish the task:

```
./submitter -t task4
```

Reflection Questions

Please respond to the following questions in the `answers/unit3answers.txt` file:

1. Why did we have to assign an external-facing load balancer to the login service?
2. As you've seen, our cloud-native architecture continues to grow, and with that growth comes significantly increased complexity. What are three

benefits and three challenges that arise from this increased complexity?

Warning

A word of caution: make sure you fully understand how to replicate this infrastructure. In the next Cloud Native project, you will build directly on top of this setup — so the more familiar you are now, the faster you'll move later.

We strongly encourage you to invest time automating the repetitive parts: provisioning SSH keys, configuring KrakenD, etc. Time spent scripting these steps now will pay off significantly in the next project.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform